DIGITAL NOTES • SOFT COMPUTING

UNIT – III

Fuzzy Decision Making & Particle Swarm Optimization

What is inside this unit

1	Fuzzy decision making – process and types
2	Bellman–Zadeh model (goals & constraints)
3	Individual, multi-person, multi-criteria, multi-stage decisions
4	Particle Swarm Optimization – concept, equations, algorithm
5	PSO example, topologies, PSO vs GA, applications
6	Solved numerical problems
7	Revision sheet & exam questions

After this unit you will be able to

- Model a decision problem with fuzzy goals and constraints (max–min principle).
- Solve simple multi-criteria and group decision problems.
- Explain how PSO works and write its update equations.
- Perform one PSO iteration by hand and compare PSO with GA.

How to use these notes

Read the **blue headings** first, study each diagram, then read the points. Yellow boxes = exam tips / key formulas. Green boxes = quick memory tricks. The last page has a one-page revision sheet and likely exam questions.

1 Fuzzy Decision Making

Decision making means choosing the best option from several alternatives. In real life, goals are vague ("cheap", "fast", "good quality") and data is uncertain. **Fuzzy decision making (FDM)** uses fuzzy sets so that goals, constraints and preferences can be expressed in **words and degrees**.



Fig 1.1 General process of fuzzy decision making

Classical decision	Fuzzy decision
Goals and constraints are crisp (cost ≤ 5000)	Goals and constraints are fuzzy (cost *should be about* 5000)
Alternative is either feasible or not	Alternative is feasible to a **degree**
Single sharp optimum	Best **compromise** with highest satisfaction degree

1.1 Types of fuzzy decision making

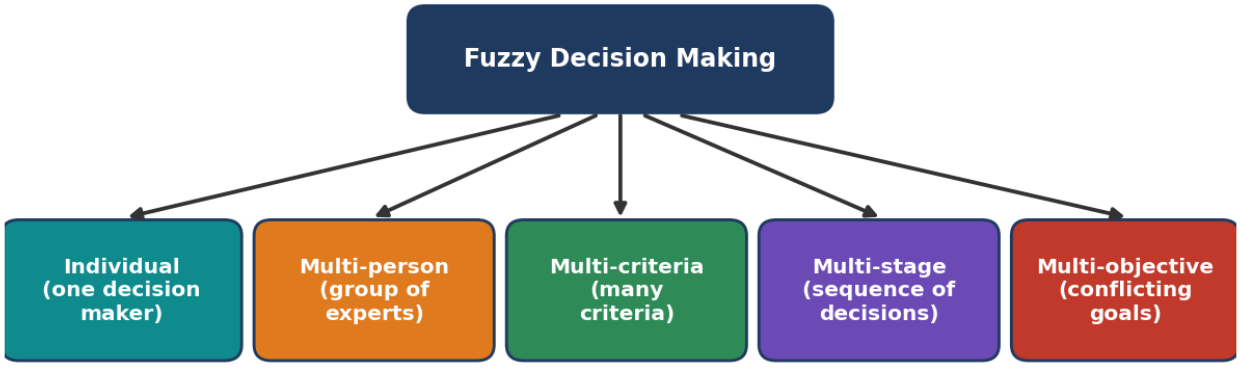


Fig 1.2 Classification of fuzzy decision problems

1.2 Bellman–Zadeh model (fuzzy goals + fuzzy constraints)

Proposed by **Bellman and Zadeh (1970)** – the base of fuzzy decision theory. A **fuzzy decision D** is the **intersection** of fuzzy goal G and fuzzy constraint C:

Key formulas

$\mu_D(x) = \min [\mu_G(x), \mu_C(x)]$ (for several goals/constraints: min over all of them)
Optimal decision: x* = the alternative with **maximum $\mu_D(x)$** ("max–min" principle)

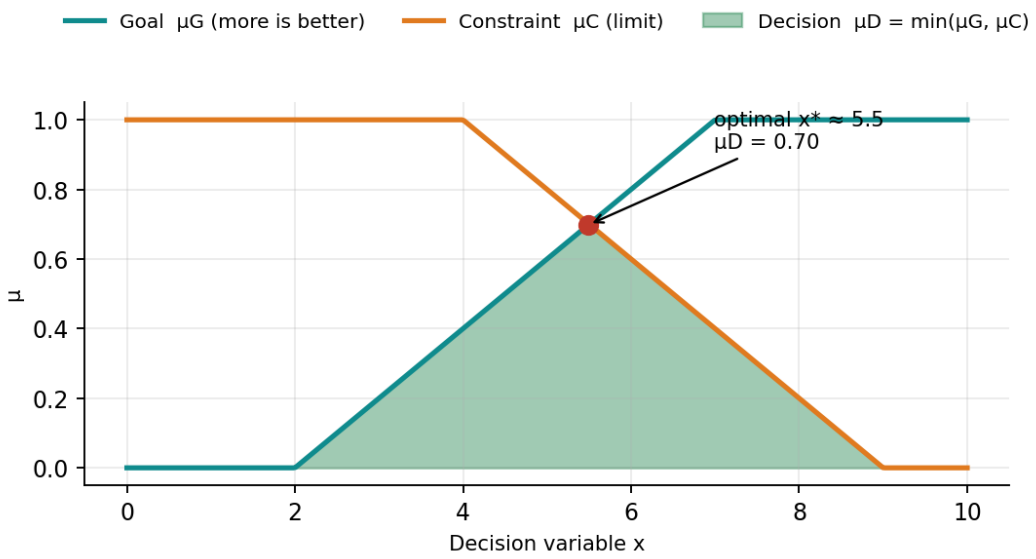


Fig 1.3 Goal and constraint conflict – the best decision is at the peak of their intersection

Worked example – choosing a laptop

Five laptops. μG = how well it meets the goal "high performance", μC = how well it satisfies the constraint "price near budget".
μG = A1: 0.9, A2: 0.7, A3: 0.5, A4: 0.6, A5: 0.3
μC = A1: 0.4, A2: 0.6, A3: 0.8, A4: 0.7, A5: 0.9
μD = min(μG, μC) = A1: 0.4, A2: 0.6, A3: 0.5, A4: 0.6, A5: 0.3
Best choice = A2 (highest μD = 0.6). Note A1 has the best performance but loses because it is too costly.

1.3 Individual decision making

- One decision maker; all alternatives **X = {x₁ … xₙ}** and their fuzzy utility / preference values are known.
- Uses **fuzzy preference relation $R(x_i, x_j)$** = degree to which x_i is preferred over x_j (R = 0.5 → indifferent).
- Ranking: for each alternative find how much it **dominates the others**: $\mu_{nd}(x_i) = 1 - \max_j [R(x_j, x_i) - R(x_i, x_j)]$; the largest value = **non-dominated best**.
- Other ways: ranking fuzzy numbers by **centroid** or **α-cut** values.

Worked example – fuzzy preference relation

Preference of x_i over x_j: R = [[0.5, 0.8, 0.6],[0.2, 0.5, 0.7],[0.4, 0.3, 0.5]] (rows = x₁, x₂, x₃).
Row-wise minimum dominance: x₁ → min(0.5, 0.8, 0.6) = 0.5; x₂ → min(0.2, 0.5, 0.7) = 0.2; x₃ → min(0.4, 0.3, 0.5) = 0.3.
Highest = x₁, so **x₁ is the best alternative**; ranking x₁ > x₃ > x₂.

1.3b Ranking fuzzy numbers

When ratings are triangular fuzzy numbers (a, b, c), convert each to one crisp score and sort. Simple method: **centroid = (a + b + c) / 3**.

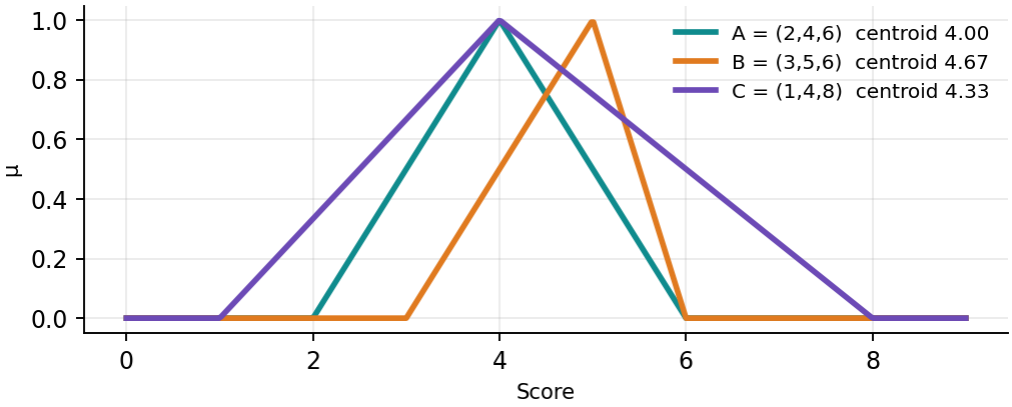


Fig 1.3b Three alternatives rated by triangular fuzzy numbers

Scores: A = 4.00, B = 4.67, C = 4.33 → **Ranking: B > C > A**. B has the highest centroid, so B is chosen.

Linguistic rating	Triangular fuzzy number
Very low	(0, 0, 0.25)
Low	(0, 0.25, 0.5)
Medium	(0.25, 0.5, 0.75)
High	(0.5, 0.75, 1)
Very high	(0.75, 1, 1)

1.4 Multi-person (group) decision making

Several experts give opinions; the system must find a **group consensus**.

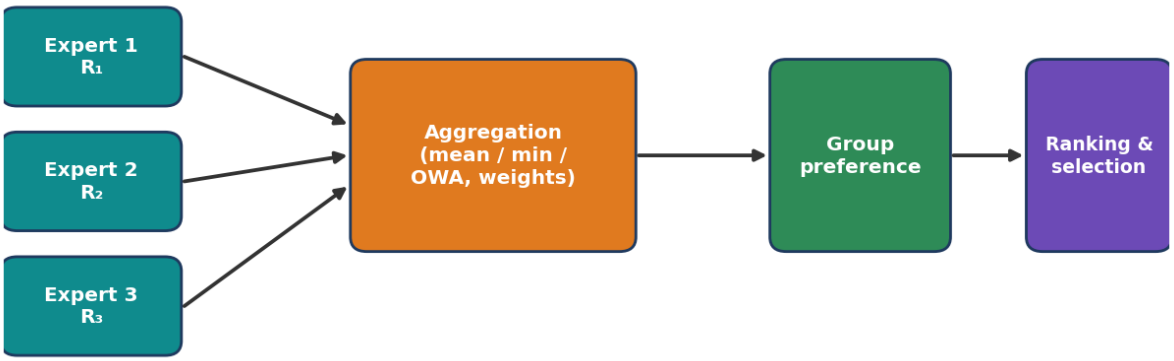


Fig 1.4 Multi-person fuzzy decision making

- Each expert gives a fuzzy preference relation R_k over the alternatives.
- **Aggregate**: arithmetic mean, weighted mean (expert importance), min (cautious) or **OWA** (ordered weighted averaging – “most experts agree”).

- Check **consensus degree**; if low, experts discuss and revise.
- Rank alternatives from the group relation and select the best.

Worked example – aggregating three experts

Experts rate alternatives A, B, C (rows E1, E2, E3): E1 = (0.9, 0.6, 0.3), E2 = (0.7, 0.8, 0.4), E3 = (0.8, 0.5, 0.6)
Mean = (0.80, 0.63, 0.43) → A is best. **Min (all must agree)** = (0.7, 0.5, 0.3) → A still best.
OWA (weights 0.5, 0.3, 0.2 on the sorted scores) for A: 0.9, 0.8, 0.7 → 0.5×0.9 + 0.3×0.8 + 0.2×0.7 = **0.83**

1.5 Multi-criteria decision making (MCDM)

The choice depends on **many criteria** (cost, quality, delivery…) with **different importance (weights)**. Ratings and weights are fuzzy / linguistic.

Step	Action
1	List alternatives and criteria; give each criterion a weight w_j ($\Sigma w = 1$)
2	Rate every alternative on every criterion → membership value μ_{ij} (from words like *good, fair*)
3	Aggregate: **weighted sum** $\Sigma\, w_j\mu_{ij}$ or **weighted min** $\min(\mu_{ij}^{w_j})$
4	Rank by final score; pick the highest

Worked example – supplier selection

Weights: Cost 0.5, Quality 0.3, Delivery 0.2
P = (0.8, 0.6, 0.7) → 0.5×0.8 + 0.3×0.6 + 0.2×0.7 = **0.72**
Q = (0.5, 0.9, 0.8) → 0.25 + 0.27 + 0.16 = **0.68**
R = (0.9, 0.4, 0.5) → 0.45 + 0.12 + 0.10 = **0.67**
Ranking: Supplier P > Supplier Q > Supplier R – Supplier P is selected.

1.5b Fuzzy TOPSIS in brief

- Build the decision matrix of fuzzy ratings and normalise it; multiply by weights.
- Find the **ideal best** (all criteria best) and **ideal worst** solutions.
- Compute distance of each alternative from both: d^+ (to best) and d^- (to worst).
- **Closeness coefficient CC = d^- / (d^+ + d^-)** – the alternative with the largest CC is selected.

1.6 Multi-stage and multi-objective decisions

- **Multi-stage**: decisions in sequence, each affecting the next (fuzzy dynamic programming). Final μ_D = min over all stages.
- **Multi-objective**: conflicting objectives (maximise profit, minimise pollution). Each objective becomes a fuzzy goal; the max–min compromise is chosen.

1.6b Case study – fuzzy loan approval

A bank must decide whether to approve a loan. Strict rules ("income > 30000 AND age < 55") reject good customers who are just outside the limit. A fuzzy system gives **partial credit**.

Criterion	Fuzzy terms	Weight
Income	Low / Medium / High	0.4
Credit history	Poor / Fair / Good	0.35
Existing debt	Light / Heavy	0.25

- Applicant: income = High (0.8), history = Fair (0.6), debt = Light (0.7).
- Score = 0.4×0.8 + 0.35×0.6 + 0.25×0.7 = 0.32 + 0.21 + 0.175 = **0.705**.
- Decision rule: score ≥ 0.65 → **approve**; 0.45–0.65 → refer to manager; below 0.45 → reject. Result: **approve**.

1.7 Applications, advantages and limits

- **Applications:** supplier / project selection, medical diagnosis choice, investment portfolio, site selection, risk assessment, e-commerce recommendation.
- **Advantages:** uses natural language; handles conflicting goals; accepts expert opinion directly.
- **Limitations:** results depend on chosen MFs / weights; many experts or criteria make computation long.

Exam tip

Always write the **max–min principle**: decision = intersection of goals and constraints = **min**; best alternative = **max** of that.

2 Particle Swarm Optimization (PSO)

PSO (Kennedy & Eberhart, **1995**) is a **swarm-intelligence** optimiser inspired by **flocking of birds and schooling of fish**. A swarm of **particles** flies through the search space; each remembers its own best place and also follows the best place found by the group.

- Each particle = one candidate solution, with a **position x** and a **velocity v**.
- **pbest** – best position this particle has visited. **gbest** – best position found by the whole swarm.
- Simple idea: "*I go where I was good before, and where my friends found food*."
- No crossover / mutation, no derivatives; only a few parameters; works for continuous problems.

2.1 Position and velocity update equations

Core equations of PSO

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t))$$
$$x_i(t+1) = x_i(t) + v_i(t+1)$$

Symbol	Meaning	Typical value
w	Inertia weight – keeps old direction; large = explore, small = exploit	0.9 → 0.4 (decreasing)
c_1	Cognitive coefficient – pull towards own best	1.5 – 2.0
c_2	Social coefficient – pull towards swarm best	1.5 – 2.0
r_1, r_2	Random numbers in [0, 1] – give randomness	new every step
Swarm size	Number of particles	20 – 50
v_{max}	Limit on velocity to avoid flying out	10–20% of range

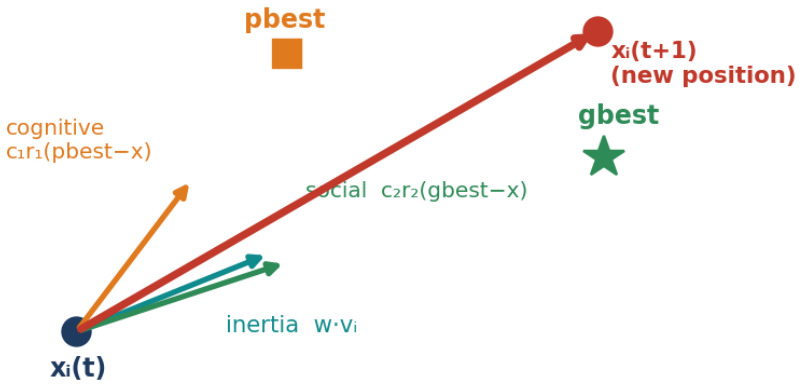


Fig 2.1 New velocity = inertia + cognitive pull + social pull

2.2 PSO algorithm (flowchart)

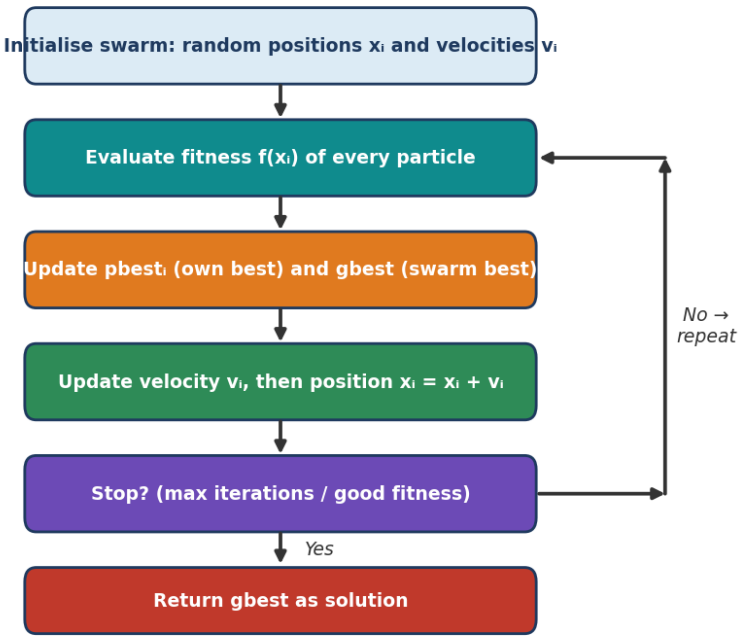


Fig 2.2 Flowchart of PSO

Algorithm PSO
1. for each particle i: random x_i, v_i; $pbest_i = x_i$
2. gbest = best of all pbest
3. repeat until stopping criterion:
   for each particle i:
     update v_i and x_i using the equations
     if $f(x_i)$ better than $f(pbest_i)$: $pbest_i = x_i$
     if $f(pbest_i)$ better than f(gbest): $gbest = pbest_i$
4. output gbest

2.3 Worked numerical example (one particle, one step)

Given

x = 4.0, v = 1.0, pbest = 3.0, gbest = 2.0; w = 0.7, $c_1 = c_2 = 1.5$; $r_1 = 0.4$, $r_2 = 0.6$
v(t+1) = 0.7×1.0 + 1.5×0.4×(3.0−4.0) + 1.5×0.6×(2.0−4.0) = 0.70 + (-0.60) + (-1.80) = **-1.70**
x(t+1) = 4.0 + (-1.70) = **2.30**
The particle moves closer to both pbest (3) and gbest (2).

2.4 Swarm behaviour in action

The figure shows 20 particles minimising $f(x, y) = x^2 + y^2$ (minimum at the ★). They start scattered and **converge** to the optimum within a few tens of iterations.

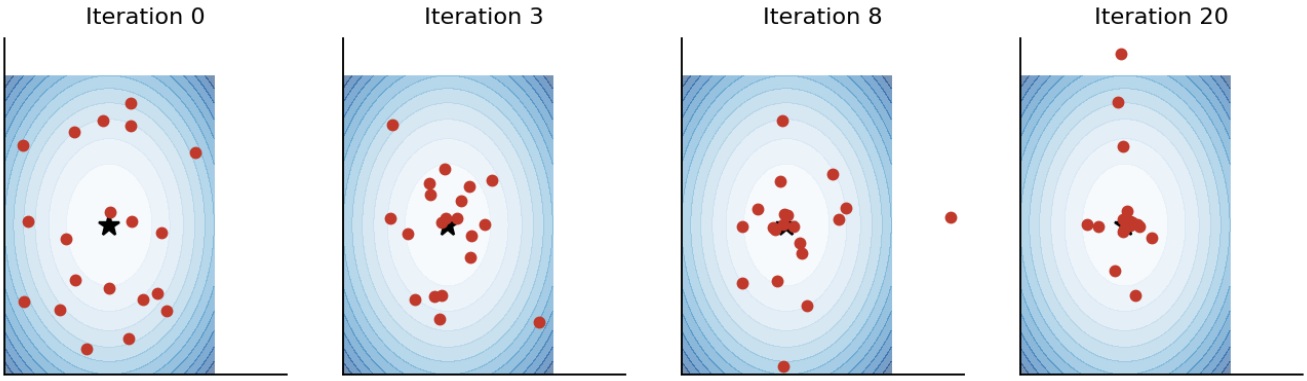


Fig 2.3 Swarm positions at iterations 0, 3, 8 and 20 (simulation)

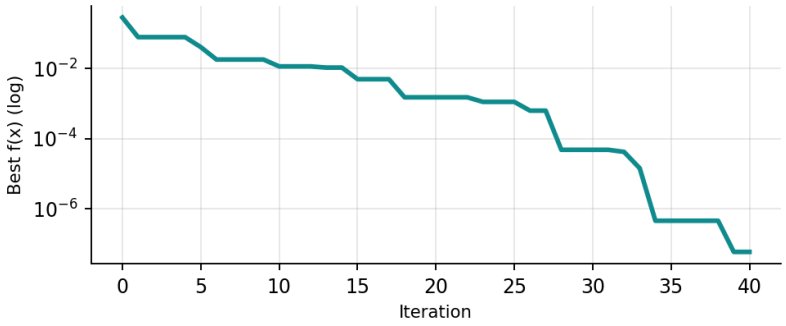


Fig 2.4 Convergence curve – best fitness falls rapidly

2.5 Neighbourhood topologies

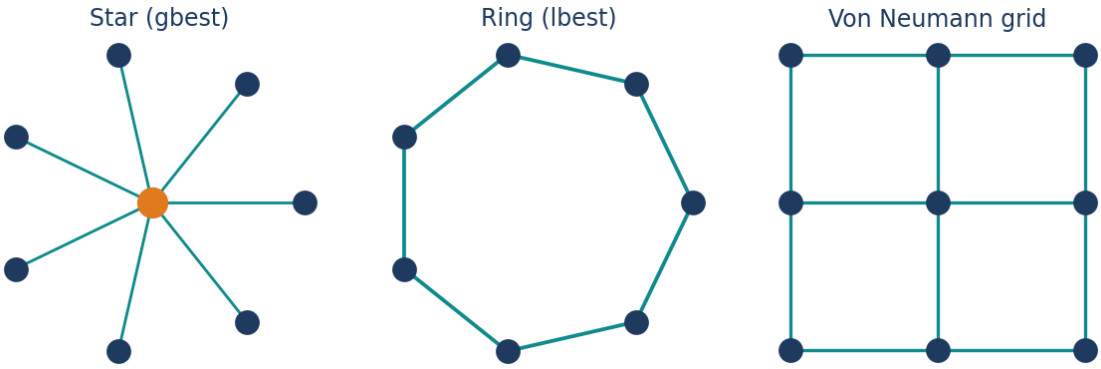


Fig 2.5 Who talks to whom: gbest (star), lbest (ring), grid

- **gbest (star):** every particle follows the global best → fast but may get trapped (premature convergence).
- **lbest (ring):** each particle follows only its neighbours → slower but explores better, avoids local optima.

2.5b Effect of inertia weight

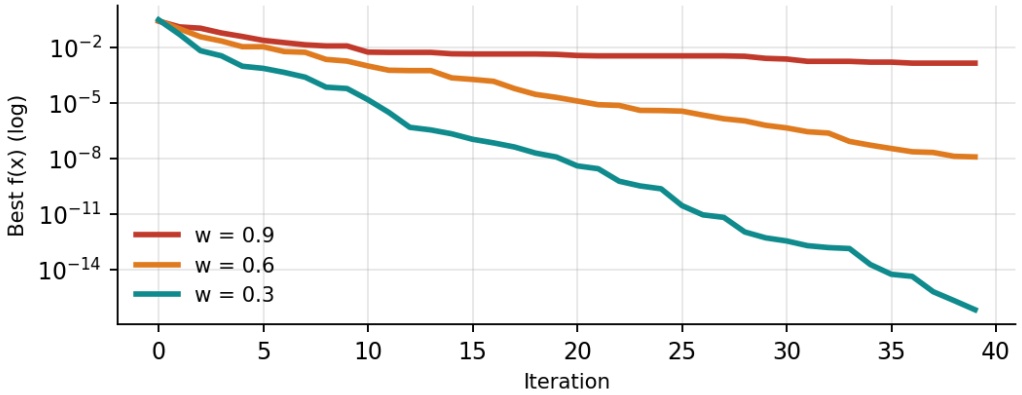


Fig 2.6 Average convergence for different inertia weights (simulation over 8 runs)

- **w large (0.9):** particles keep moving – good exploration, but slow and may overshoot.
- **w small (0.3):** fast settling – good exploitation, but risk of local optimum.
- Best practice: **start high and reduce linearly** (e.g. 0.9 → 0.4).

2.5c Important PSO variants

Variant	Idea
Constriction-factor PSO	A factor χ multiplies the whole velocity to guarantee convergence
Binary PSO	Velocity becomes a probability (sigmoid) that a bit is 1 – for discrete problems
Adaptive / dynamic PSO	w, c_1, c_2 change automatically during the run
Hybrid PSO-GA / PSO-DE	Adds mutation or crossover to keep diversity

2.5d Using PSO to tune a fuzzy controller

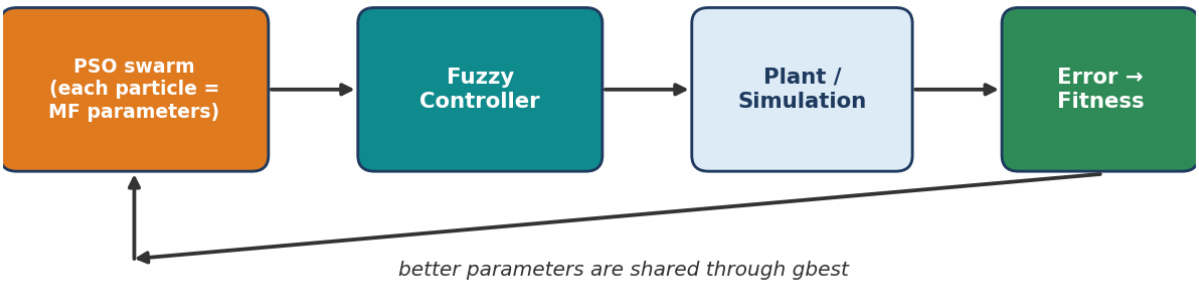


Fig 2.7 PSO searches the best membership-function parameters by minimising control error

2.6 PSO vs Genetic Algorithm

Point	PSO	GA
Inspiration	Bird flocking / social behaviour	Natural evolution
Operators	Velocity & position update	Selection, crossover, mutation
Memory	Particles remember pbest and gbest	Only the current population
Representation	Real numbers (mostly)	Binary / real / permutation
Parameters	Few (w, c_1, c_2)	More (pc, pm, selection type)
Speed	Usually faster convergence	Slower but more diverse

2.6b Practical issues in PSO

Issue	Remedy
Particles leave the search range	Clamp position to the bounds or reflect the particle back
Velocity explosion	Limit with v_{max} or use constriction factor
Premature convergence	Use lbest topology, decreasing w, or add mutation
When to stop?	Max iterations, fitness below a threshold, or no improvement for N iterations
Choosing swarm size	20–50 is enough for most problems; larger for high dimensions

2.7 Advantages, limitations, applications

- **Advantages:** simple, few parameters, easy to code, fast, no gradient needed.
- **Limitations:** can converge prematurely to local optimum; weak for discrete problems unless modified (binary PSO); parameter sensitivity.
- **Applications:** neural-network training, fuzzy-controller tuning, power-system dispatch, feature selection, image segmentation, robot path planning, antenna design.

Memory trick

PSO = Position + Speed + Own best + Group best. "I trust my past (c_1) and my friends (c_2), but I also keep some momentum (w)."

3 Solved Numerical Problems

Problem 1 – One full PSO iteration (3 particles)

Minimise $f(x) = x^2$. Start: x = (4, −3, 2), v = (1, −1, 0.5), pbest = x, so initial **gbest = 2** (f = 4). Take w = 0.5, $c_1 = c_2 = 1.0$, $r_1 = r_2 = 0.5$. Formula: $v' = w \cdot v + c_1 r_1(pbest - x) + c_2 r_2(gbest - x)$.

Particle	x	v	pbest	new v	new x	f(new x)	new pbest
P1	4.00	1.00	4.00	-0.50	3.50	12.25	3.50
P2	-3.00	-1.00	-3.00	2.00	-1.00	1.00	-1.00
P3	2.00	0.50	2.00	0.25	2.25	5.06	2.00

Example, P2: $v' = 0.5\times(-1) + 1\times0.5\times(-3-(-3)) + 1\times0.5\times(2-(-3)) = -0.5 + 0 + 2.5 =$ **2.00**; $x' = -3 + 2 =$ **-1.00**, f = 1.00 which is better than 9, so pbest becomes -1.00.

New gbest = -1.00 (f = 1.00). The swarm has already moved much closer to the minimum at x = 0.

Problem 2 – Fuzzy decision (max–min)

Goal μG = (0.7, 0.9, 0.4), constraint μC = (0.8, 0.5, 0.6) for alternatives (a, b, c).
μD = min(μG, μC) = (0.7, 0.5, 0.4) → **choose alternative a** (μD = 0.7).

Problem 3 – Weighted multi-criteria

Weights (0.6, 0.4). Alt X = (0.5, 0.9) → 0.3 + 0.36 = **0.66**. Alt Y = (0.8, 0.5) → 0.48 + 0.20 = **0.68**. **Y is selected**, though X is much better on criterion 2.

Common mistakes to avoid

• Using max instead of min for the Bellman–Zadeh decision. • Forgetting that r_1, r_2 are random each step. • Updating position before velocity. • Not updating pbest/gbest after the move.

4 One-page Revision & Important Questions

Formula sheet

• Fuzzy decision: $\mu_D = \min(\mu_G, \mu_C)$; best = argmax μ_D. • MCDM score = $\Sigma\, w_j\mu_{ij}$.
• PSO: $v \leftarrow w\cdot v + c_1r_1(pbest - x) + c_2r_2(gbest - x)$; $x \leftarrow x + v$.
• Large w = exploration; small w = exploitation. • gbest = fast; lbest = diverse.

Quick self-check

Q1. Which operator gives the Bellman–Zadeh decision? → **min (intersection)**
Q2. What does c_2 represent in PSO? → **Social learning (pull towards gbest)**
Q3. Who proposed PSO? → **Kennedy & Eberhart, 1995**
Q4. Name two problems of PSO. → **Premature convergence, poor on discrete problems**

Likely exam questions

- Explain fuzzy decision making. Describe the Bellman–Zadeh model with an example.
- Explain multi-person and multi-criteria fuzzy decision making.
- Explain PSO with algorithm / flowchart and the velocity update equation.

- Solve: compute new velocity and position of a particle (numerical).
- Compare PSO with GA; list advantages, limitations and applications of PSO.